UNIT – III

Cryptographic Hash Functions & Key Management and Distribution

CRYPTOGRAPHIC HASH FUNCTIONS

1. Message Authentication

1.1 What is Message Authentication?

Message authentication is a mechanism or service used to verify the integrity of a message and authenticate the source of a message. It ensures that data has not been altered or that unauthorized parties have not replaced a message.

Authentication Requirements

A message authentication scheme must satisfy the following requirements:

- **Message integrity:** The receiver must be able to verify that the message was not altered during transmission.
- **Source authentication:** The receiver must be able to verify that the message was sent by the claimed source.
- **Non-repudiation:** The sender must not be able to deny sending the message.
- **Timeliness:** The receiver must be able to verify that the message was not delayed or replayed.

Functions Used for Message Authentication

Three types of functions may be used to produce an authenticator:

Type	Description	Example
Hash Function	Maps a variable-length message to a fixed-length hash value (message digest)	SHA-512, MD5
Message Encryption	The ciphertext serves as an authenticator	AES, DES
MAC (Message Authentication Code)	A function of message and a secret key producing a fixed-length value	HMAC, CMAC

1.2 Message Authentication Code (MAC)

A MAC is a small block of data generated using a secret key and a message. It provides both data integrity and authentication.

MAC Generation Process

1. Sender computes: MAC = C(K, M), where K is the secret key and M is the message.

2. Message M and MAC are sent to the receiver.
3. Receiver recomputes MAC' = C(K, M) using the same key.
4. If MAC == MAC', the message is authentic and unaltered.

> **Note:** Unlike encryption, MAC does not provide confidentiality. Its sole purpose is to authenticate the message source and ensure message integrity.

2. Secure Hash Algorithm – SHA-512

2.1 Overview

SHA-512 is part of the SHA-2 family of cryptographic hash functions, designed by the National Security Agency (NSA) and published by NIST in 2001. It produces a 512-bit (64-byte) hash value, typically rendered as a 128-digit hexadecimal number.

Property	Value
Message Digest Size	512 bits (64 bytes)
Message Size	< 2^128 bits
Block Size	1024 bits
Word Size	64 bits
Number of Steps	80 rounds
Security Level	256-bit (collision resistance)

2.2 SHA-512 Algorithm Steps

Step 1: Append Padding Bits

The original message is padded so that its length is congruent to 896 mod 1024 (i.e., 128 bits short of a multiple of 1024). Padding always begins with a '1' bit followed by '0' bits.

Step 2: Append Length

A 128-bit representation of the original message length (before padding) is appended to the padded message, making the total length a multiple of 1024 bits.

Step 3: Initialize Hash Buffer

SHA-512 uses eight 64-bit registers (a, b, c, d, e, f, g, h) initialized with specific constant values derived from the fractional parts of the square roots of the first eight prime numbers.

Initial Hash Values (H0–H7)

H0 = 6a09e667f3bcc908 H1 = bb67ae8584caa73b

H2 = 3c6ef372fe94f82b H3 = a54ff53a5f1d36f1
H4 = 510e527fade682d1 H5 = 9b05688c2b3e6c1f
H6 = 1f83d9abfb41bd6b H7 = 5be0cd19137e2179

Step 4: Process Message in 1024-bit Blocks

Each 1024-bit block is processed through 80 rounds. Each round uses a round constant Kt (derived from cube roots of first 80 primes) and a 64-bit word Wt from the message schedule.

- **Message Schedule:** 16 words are taken directly from the block; remaining 64 words are computed using bitwise operations.
- **Round Function:** Involves six logical functions: Ch, Maj, Sigma0, Sigma1, sigma0, sigma1 operating on 64-bit words.

Step 5: Output

After all blocks are processed, the final 512-bit message digest is obtained by concatenating the eight 64-bit hash values: H0 || H1 || H2 || H3 || H4 || H5 || H6 || H7.

2.3 SHA-512 Logical Functions

Function	Formula	Purpose
Ch(e,f,g)	(e AND f) XOR (NOT e AND g)	Choice based on e
Maj(a,b,c)	(a AND b) XOR (a AND c) XOR (b AND c)	Majority of a,b,c
Sigma-0(a)	ROTR28(a) XOR ROTR34(a) XOR ROTR39(a)	Mix of register a
Sigma-1(e)	ROTR14(e) XOR ROTR18(e) XOR ROTR41(e)	Mix of register e
sigma-0(x)	ROTR1(x) XOR ROTR8(x) XOR SHR7(x)	Message schedule
sigma-1(x)	ROTR19(x) XOR ROTR61(x) XOR SHR6(x)	Message schedule

> **Note:** ROTR = Right Rotation, SHR = Right Shift. All operations are performed on 64-bit words in SHA-512.

3. HMAC – Hash-based Message Authentication Code

3.1 Introduction

HMAC is a specific type of MAC involving a cryptographic hash function and a secret key. It can be used with any iterated cryptographic hash function such as SHA-512 or MD5. HMAC is defined in RFC 2104 and is widely used in TLS/SSL and IPSec protocols.

3.2 HMAC Algorithm

Given a hash function H, message M, and key K:

1. Determine the block length b and output length n of the hash function.
2. If K is longer than b, set K = H(K); otherwise, pad K with zeros on the right to make it b bits long.
3. Compute K XOR ipad (inner padding: 0x36 repeated b/8 times).
4. Append M to the result: (K XOR ipad) || M.
5. Apply H to produce inner hash: H((K XOR ipad) || M).
6. Compute K XOR opad (outer padding: 0x5C repeated b/8 times).
7. Append inner hash: (K XOR opad) || H((K XOR ipad) || M).
8. Apply H again: HMAC = H((K XOR opad) || H((K XOR ipad) || M)).

HMAC Formula

HMAC(K, M) = H[(K+ opad) || H[(K+ ipad) || M]]

Where '+' denotes XOR operation and '||' denotes concatenation.

3.3 Security of HMAC

- **Provable security:** HMAC's security is directly tied to the underlying hash function's properties.
- **Resistance to length extension attacks:** Unlike plain hash MACs, HMAC is immune to length extension attacks due to the double hashing.
- **Key flexibility:** Keys of any length up to the block size can be used.

HMAC Variant	Security Bits
HMAC-MD5	128 bits (considered weak today)
HMAC-SHA1	160 bits
HMAC-SHA256	256 bits
HMAC-SHA512	512 bits (strongest)

4. CMAC – Cipher-based Message Authentication Code

4.1 Overview

CMAC (also known as OMAC) is a MAC algorithm based on a block cipher such as AES. It was recommended by NIST in SP 800-38B and addresses weaknesses in the earlier CBC-MAC scheme.

4.2 CMAC vs CBC-MAC

CBC-MAC	CMAC
Insecure for variable-length messages	Secure for variable-length messages
Requires separate length block	Handles variable length inherently
Uses single key	Uses key + two derived subkeys (K1, K2)
Padding not well-defined	Uses well-defined 10* padding

> **Note:** CMAC with AES-128 provides 128-bit security and is used extensively in banking, IoT security, and government applications.

DIGITAL SIGNATURES

5. Digital Signatures

5.1 Introduction

A digital signature is a mathematical scheme for verifying the authenticity and integrity of digital messages or documents. It provides the equivalent of a handwritten signature or stamped seal in the digital world, but with far greater inherent security.

5.2 Properties of Digital Signatures

- **Authentication:** The receiver can verify the claimed identity of the sender.
- **Integrity:** The receiver can verify the message has not been altered since signing.
- **Non-repudiation:** The sender cannot deny having sent the signed message.
- **Unforgeability:** No one except the signer can produce a valid signature.

5.3 Digital Signature Process

Signing Process (Sender)

9. Compute message hash: h = H(M)
10. Sign the hash using private key (KRa): S = E(KRa, h)
11. Send message M along with signature S

Verification Process (Receiver)

12. Receive M and S
13. Compute hash of received message: h' = H(M)
14. Decrypt signature using sender's public key (KUa): h = D(KUa, S)
15. If h' == h, signature is valid — message is authentic and unmodified

5.4 Digital Signature Algorithms (DSA)

6. ElGamal Digital Signature Scheme

6.1 Background

The ElGamal signature scheme was proposed by Taher ElGamal in 1985. It is based on the discrete logarithm problem (DLP) and forms the mathematical basis for the Digital Signature Algorithm (DSA).

6.2 Key Generation

16. Choose a large prime p and a generator g of Z*_p (the multiplicative group modulo p).
17. Select a random private key x, where 1 < x < p-1.
18. Compute public key y = g^x mod p.
19. Public key: (p, g, y). Private key: x.

6.3 Signature Generation

To sign a message M:

20. Compute hash value H(M).
21. Choose a random k with gcd(k, p-1) = 1 (k must be kept secret and unique per message).
22. Compute r = g^k mod p.
23. Compute s = k^(-1) * (H(M) - x*r) mod (p-1).
24. The signature is the pair (r, s).

ElGamal Signature

Signature: (r, s) where:

r = g^k mod p

s = k^-1 * (H(M) - x*r) mod (p-1)

6.4 Signature Verification

To verify signature (r, s) for message M with public key (p, g, y):

25. Compute v1 = g^H(M) mod p.
26. Compute v2 = (y^r * r^s) mod p.
27. The signature is valid if and only if v1 == v2.

KEY MANAGEMENT AND DISTRIBUTION

7. Key Management and Distribution

7.1 Introduction

Key management encompasses all aspects of key handling: generation, distribution, storage, retirement, and destruction. Secure key distribution is one of the most challenging aspects of practical cryptography.

7.2 Symmetric Key Distribution Using Symmetric Encryption

In symmetric encryption, both parties must share a secret key. Key distribution can be achieved in several ways:

Methods of Symmetric Key Distribution

- **Physical delivery:** A key is physically delivered to communicating parties — impractical at scale.
- **Key Distribution Center (KDC):** A trusted third party (KDC) shares a unique key with each participant. The KDC creates session keys on demand.
- **Key Hierarchy:** Master keys encrypt session keys; session keys encrypt data. This limits exposure of any single key.

KDC-Based Key Distribution (Needham-Schroeder Protocol)

28. A requests session key for communication with B from KDC.
29. KDC responds with: [KS, IDA, E(Kb, [KS, IDA])] encrypted with Ka.
30. A decrypts using Ka to get session key KS and a 'ticket' for B.
31. A sends the ticket E(Kb, [KS, IDA]) to B.
32. B decrypts the ticket using Kb to get session key KS.
33. A and B now communicate securely using KS.

7.3 Symmetric Key Distribution Using Asymmetric Encryption

Asymmetric (public-key) cryptography can be used to securely distribute symmetric session keys, combining the convenience of asymmetric methods with the efficiency of symmetric encryption.

Simple Protocol

34. A obtains B's public key PUb.
35. A generates a secret key KS and encrypts it: E(PUb, KS).
36. A sends the encrypted KS to B.

37. B decrypts using private key PRb: D(PRb, E(PUb, KS)) = KS.
38. Both A and B now share symmetric key KS.

> **Note:** This approach is vulnerable to man-in-the-middle attacks if public keys are not properly authenticated. This motivates the need for digital certificates and PKI.

Authenticated Key Exchange

39. A generates KS, encrypts and signs: E(PUb, [KS || IDA || N1])
40. B decrypts, verifies A's identity, generates N2.
41. B encrypts response with KS: E(KS, [N1 || N2])
42. A decrypts, confirms freshness via N1, sends E(KS, N2) to confirm.

8. Distribution of Public Keys

8.1 The Problem

In a public-key system, anyone can encrypt a message for a user if they have the user's public key. However, how does one obtain an authentic public key? This is the key distribution problem for asymmetric cryptography.

8.2 Methods for Public Key Distribution

1. Public Announcement

Users broadcast their public keys to the community. Simple but insecure — anyone can forge a public key announcement impersonating another user.

2. Publicly Available Directory

A trusted entity maintains a directory mapping names to public keys. Better than announcement, but still vulnerable if the directory is compromised.

3. Public Key Authority

A central authority holds a database of public keys. Users request keys from this authority. Each user knows the authority's public key. Bottleneck at the authority; requires real-time connection.

4. Public Key Certificates (Most Common)

A Certificate Authority (CA) digitally signs a certificate binding a user's identity to their public key. Anyone with the CA's public key can verify the certificate without contacting the CA.

Method	Security Level
Public Announcement	Low — forgery possible
Public Directory	Medium — directory must be trusted
Public Key Authority	High — but requires online authority

Certificates (X.509)	High — offline verification possible

9. Kerberos Authentication Service

9.1 Overview

Kerberos is a network authentication protocol developed at MIT in the 1980s (Project Athena). It provides strong authentication for client/server applications using symmetric-key cryptography and a trusted third party — the Key Distribution Center (KDC).

9.2 Kerberos Components

Component	Role
Client (C)	User/workstation requesting service
Server (S)	Service the client wants to access (e.g., file server)
Authentication Server (AS)	Part of KDC; authenticates users
Ticket Granting Server (TGS)	Part of KDC; issues service tickets
Ticket	Encrypted credential proving identity to a server
Authenticator	Time-stamped proof that client holds the session key

9.3 Kerberos Authentication Protocol

Phase 1: Initial Authentication (Client ↔ AS)

43. Client sends user ID and TGS ID to AS (in plaintext).
44. AS returns: Ticket-Granting Ticket (TGT) = E(Ktgs, [IDc, ADc, IDtgs, TS1, Lifetime1, Kc,tgs]) and session key E(Kc, Kc,tgs).
45. Client decrypts using password-derived key Kc to get session key Kc,tgs.

Phase 2: Ticket Granting (Client ↔ TGS)

46. Client sends TGT + Authenticator (containing IDc, timestamp, encrypted with Kc,tgs) + IDv (desired service).
47. TGS decrypts TGT to get Kc,tgs; decrypts Authenticator to verify identity and freshness.
48. TGS returns Service Ticket E(Kv, [IDc, ADc, IDv, TS2, Lifetime2, Kc,v]) and E(Kc,tgs, Kc,v).

Phase 3: Service Access (Client ↔ Server)

49. Client sends Service Ticket + new Authenticator (encrypted with Kc,v).

50. Server decrypts ticket to get Kc,v; verifies Authenticator.
51. Server confirms identity to client by returning E(Kc,v, TS2 + 1) (optional mutual auth).
52. Client and server communicate using session key Kc,v.

9.4 Kerberos Versions

Version	Key Features
Kerberos v4	Original; DES-only; limited to single realm; deprecated
Kerberos v5 (RFC 4120)	Current; supports multiple encryption types; cross-realm; ticket renewal; pre-authentication

9.5 Strengths and Weaknesses

- **Strengths:** Mutual authentication, single sign-on, centralized access control, replay attack prevention via timestamps.
- **Weaknesses:** Single point of failure (KDC), requires time synchronization, password guessing attacks on initial ticket, complex setup.

10. X.509 Authentication Service

10.1 Introduction

X.509 is an ITU-T standard for defining the format of public key certificates. It was originally published in 1988 as part of the X.500 directory service standards and has become the most widely used certificate format in security protocols such as TLS/SSL, S/MIME, and IPSec.

10.2 X.509 Certificate Format (v3)

Field	Description
Version	Certificate format version (v1, v2, v3)
Serial Number	Unique integer assigned by the CA
Signature Algorithm	Algorithm used to sign the certificate (e.g., SHA256withRSA)
Issuer	Distinguished name (DN) of the CA that issued the certificate
Validity Period	Not Before and Not After dates
Subject	Distinguished name of the entity the certificate identifies

Subject Public Key Info	Algorithm and public key value
Issuer Unique ID (v2+)	Optional bit string uniquely identifying the issuer
Subject Unique ID (v2+)	Optional bit string uniquely identifying the subject
Extensions (v3 only)	Additional attributes (key usage, SAN, CRL distribution points)
Signature	CA's digital signature over all preceding fields

10.3 X.509 Certificate Chain

Certificates are organized in a hierarchical chain of trust:

- **Root CA Certificate:** Self-signed certificate of the root authority; implicitly trusted.
- **Intermediate CA Certificate:** Issued by root CA; used to sign end-entity certificates (better security practice).
- **End-Entity Certificate:** Issued to a user, server, or device; contains the public key being certified.

10.4 Authentication Procedures Using X.509

One-Way Authentication

A → B: A sends certificate, timestamp, B's identifier, signature, and optionally a session key. B verifies A's signature using A's public key from the certificate.

Two-Way Authentication

Extends one-way: B also sends its certificate and signature back, including A's earlier token. Both parties authenticate each other.

Three-Way Authentication

Adds an extra message from A to B to avoid reliance on synchronized clocks. Uses challenge-response nonces instead of timestamps.

10.5 Certificate Revocation

- **CRL (Certificate Revocation List):** Periodically published list of revoked certificates with serial numbers and reason codes.
- **OCSP (Online Certificate Status Protocol):** Real-time query to check if a specific certificate is revoked; more efficient than CRL for large deployments.

11. Public Key Infrastructure (PKI)

11.1 Definition

Public Key Infrastructure (PKI) is the set of roles, policies, hardware, software, and procedures needed to create, manage, distribute, use, store, and revoke digital certificates and manage public-key encryption. PKI binds public keys to entities through trusted authorities.

11.2 PKI Components

Component	Function
Certificate Authority (CA)	Issues and signs certificates; root of trust
Registration Authority (RA)	Verifies identity before CA issues certificates; offloads CA workload
Certificate Repository	Stores issued certificates; LDAP directories or web servers
Certificate Revocation	CRL or OCSP services for revocation checking
Key Archival Server	Optionally stores encrypted private keys for recovery
End Entities	Users, servers, devices holding certificates

11.3 PKI Trust Models

1. Single CA (Monopoly Model)

One root CA trusted by all entities. Simple but represents a single point of failure. Suitable for small closed communities.

2. Hierarchical CA

Root CA at top; subordinate CAs below. Root delegates trust to intermediate CAs. Most common model (e.g., public internet PKI with browsers trusting root CAs).

3. Web of Trust (Decentralized)

Used by PGP. No central authority; users sign each other's keys. Trust is built transitively through peer endorsements.

4. Cross-Certification

Two CAs issue certificates to each other, enabling clients of one CA to trust certificates issued by another. Used in federated enterprise environments.

11.4 Certificate Lifecycle

53. **Request:** Entity generates key pair, submits certificate signing request (CSR) to RA.
54. **Verification:** RA verifies identity; sends approved request to CA.
55. **Issuance:** CA signs and issues X.509 certificate.
56. **Distribution:** Certificate published to repository; sent to entity.
57. **Use:** Entity presents certificate in SSL/TLS, email, VPN, etc.

58. **Renewal:** Before expiry, entity requests a new certificate.
59. **Revocation:** If key is compromised, CA revokes certificate; updates CRL/OCSP.

11.5 PKI Applications

- TLS/SSL certificates for HTTPS websites (e.g., browser padlock)
- S/MIME for secure email encryption and digital signatures
- Code signing certificates to verify software authenticity
- VPN authentication using device certificates
- Smart card / PIV authentication for government employees
- Document signing in Adobe PDF, Microsoft Office

Key Point for Exams

PKI does NOT solve all key management problems — it still requires a trusted root CA. The security of the entire PKI depends on the security of the root CA's private key.

QUICK REVISION SUMMARY

Topic	Key Concept	Use / Application
Hash Functions	One-way function; fixed output from variable input	Integrity checking, digital signatures
SHA-512	512-bit digest; 80 rounds; 1024-bit blocks	Highly secure hashing standard
MAC	C(K, M) = authentication tag using shared key	Data integrity + source authentication
HMAC	H((K XOR opad) || H((K XOR ipad) || M))	TLS, IPSec, API authentication
CMAC	AES-based MAC; uses subkeys K1/K2	Banking, IoT, government protocols
Digital Signature	E(PR, H(M)); verified with public key	Non-repudiation, software signing
ElGamal Signature	Based on DLP; signature (r, s)	Basis for DSA
Kerberos	KDC + TGT + Service Ticket protocol	Network single sign-on (Windows AD)
X.509	Standard format for public key certificates	TLS, S/MIME, code signing
PKI	Infrastructure for managing certificates and keys	HTTPS, VPN, email security, smart cards

Important Formulas

Item	Formula / Value
SHA-512 output	512 bits (64 bytes)
SHA-512 block size	1024 bits
HMAC	H[(K ⊕opad) || H[(K ⊕ipad) || M]]
ElGamal r	r = g^k mod p
ElGamal s	s = k^-1 * (H(M) - x*r) mod (p-1)
ElGamal verify	g^H(M) mod p == y^r * r^s mod p
Kerberos ticket	E(Kserver, [IDc, ADc, IDv, TS, Lifetime, Kcs])